

Hands-on Planning: Online Motion Planning Lab

Date: March 26, 2026

Author: Duc Phu Nguyen, Ba Quang Huy Do, Mohammad
Haadi Akhter



Universitat de Girona
Escola Politècnica Superior

Contents

1	Introduction	2
2	Implementation	2
2.1	Part 1: Low-Level Motion Control Node	3
2.1.1	Methodology	3
2.1.2	System Architecture	3
2.1.3	Pseudocode	4
2.2	Part 2: Environment Mapping	5
2.2.1	Methodology	5
2.2.2	System Architecture and Frame Configuration	5
2.2.3	Mapping Logic	6
2.3	Part 3: Path Planning and Control	8
2.3.1	Methodology	8
2.3.2	System Configuration	8
2.3.3	System Architecture	9
2.3.4	Algorithm Pseudocode	10
3	Results and Discussion	11
3.1	Part 1: Low-Level Motion Control Node	11
3.1.1	Controller Parameter Configuration	11
3.1.2	Execution and Deployment Instructions	11
3.2	Part 2: Environment Mapping	13
3.3	System Configuration: Coordinate Frames	13
3.3.1	Coordinate Frame Analysis (ENU vs. NED)	13
3.3.2	Execution and Deployment Instructions	13
3.4	Mapping Result	14
3.5	Part 3: Path Planning and Navigation	15
3.5.1	Parameter Configuration	15
3.5.2	Manual: Operating the Online Motion Planning System	16
3.5.3	Simulation Results	16
3.5.4	Real World Results	17
4	Conclusion	19

1 Introduction

This report documents the implementation of autonomous navigation functionalities on a TurtleBot mobile robot within the ROS 2 Jazzy ecosystem. The primary objective of this project is to develop a robust system capable of low-level motion control, environment mapping, and online path planning.

The project is structured into three main phases:

- **Phase 1:** Development of a proportional controller to drive the robot to user-defined goal poses using odometry data.
- **Phase 2:** Construction of a 2D map of the environment using a probabilistic occupancy grid framework and LiDAR sensor data.
- **Phase 3:** Integration of sampling-based planning algorithms, such as RRT or RRT*, to enable obstacle avoidance and real-time replanning in dynamic environments.

By the end of this laboratory, the TurtleBot will be able to incrementally build a map of its surroundings while simultaneously exploiting that information to reach targets defined in RViz safely. The following sections detail the methodology, system architecture, and experimental results for each phase of the implementation.

2 Implementation

2.1 Part 1: Low-Level Motion Control Node

2.1.1 Methodology

In this part, we implement a low-level motion controller for the TurtleBot based on a **uni-cycle model control law**. The objective is to drive the robot from its current pose (x, y, θ) to a target position (x_g, y_g) received from RViz. The controller calculates the position error in the world frame as:

$$inc_x = x_g - x$$

$$inc_y = y_g - y$$

The steering logic follows a "**rotate-then-translate**" strategy to ensure smooth and accurate motion. First, the angular velocity w is computed proportional to the heading error using $w = k_w \cdot (\text{atan2}(inc_y, inc_x) - \theta)$. To ensure the shortest turn, the error is normalized within $[-\pi, \pi]$. The linear velocity v is only applied when the heading error is below a specific threshold $\omega_{threshold}$:

- If $|w|[\text{cite}_{start}] < \omega_{threshold}$, then $v = k_v \cdot \sqrt{inc_x^2 + inc_y^2}$.
- Otherwise, $v = 0$ to allow the robot to rotate in place.

2.1.2 System Architecture

The motion control system is encapsulated in a ROS 2 node named `control_tb.py` (represented as the **ControlTurtlebot** node in Figure 4). The architecture follows a modular callback-based design:

- **Data Acquisition:** The node subscribes to `nav_msgs/Odometry` via the `/turtlebot/odom` topic to extract the robot's current (x, y) position and orientation (yaw). It also listens to `geometry_msgs/PoseStamped` on the `/goal_pose` topic to set the target coordinates.
- **Internal Processing:** Data from these topics are stored internally as robot state and goal points.
- **Command Generation:** A periodic timer triggers the `control_loop()` function, which computes the P-Gains based on the control law.
- **Actuation:** The resulting velocity commands are published as `geometry_msgs/Twist` messages to the `/turtlebot/cmd_vel` topic to drive the robot.

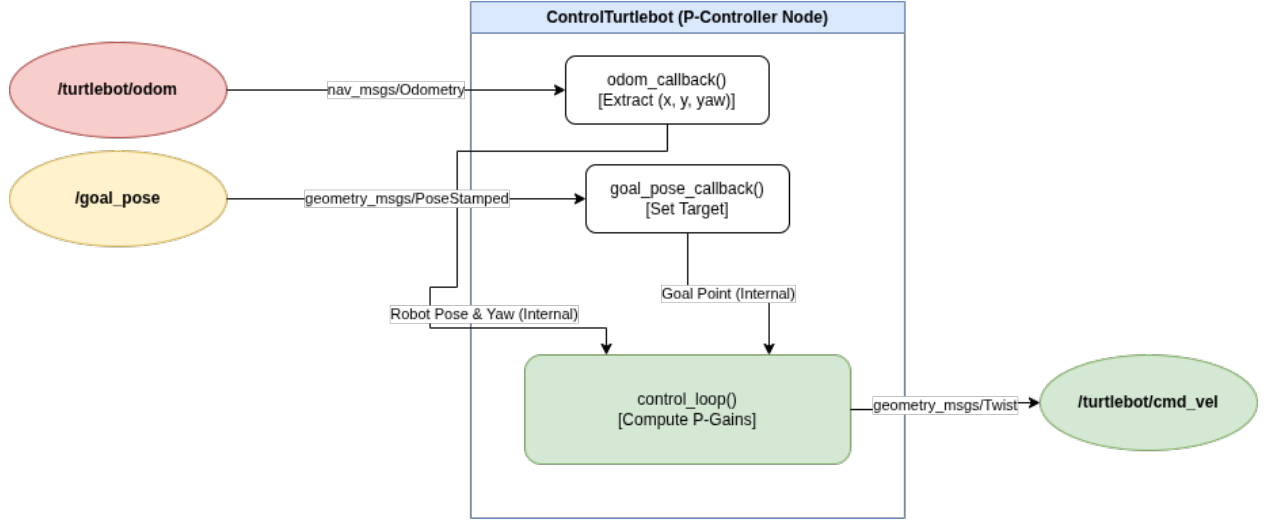


Figure 1: ROS 2 Node Architecture: Interaction between subscribers, internal control logic, and the velocity publisher.

2.1.3 Pseudocode

The control logic is implemented as follows:

Algorithm 1 Low-Level Proportional Controller for Mobile Robot

Require: Current pose (x, y, θ) from `/odom`, goal pose (x_g, y_g) , gains k_v, k_w , acceptance radius R , thresholds ω_{thresh}, v_{max}

Ensure: Velocity commands v, ω published to `/cmd_vel`

```

0: for each time step at 10Hz do
0:    $e_d \leftarrow \sqrt{(x_g - x)^2 + (y_g - y)^2}$  {Compute distance error}
0:   if  $e_d < R$  then {Goal Reached}
0:      $v \leftarrow 0, \omega \leftarrow 0$ 
0:   else
0:      $\theta_{target} \leftarrow \text{atan2}(y_g - y, x_g - x)$ 
0:      $e_\theta \leftarrow \text{normalize}(\theta_{target} - \theta)$ 
0:      $\omega \leftarrow k_w \cdot e_\theta$  {Proportional angular control}
0:     if  $|e_\theta| < \omega_{thresh}$  then {Move forward if aligned}
0:        $v \leftarrow \min(k_v \cdot e_d, v_{max})$ 
0:     else {Rotate in place}
0:        $v \leftarrow 0$ 
0:     end if
0:   end if
0:   Publish control command:  $\text{Twist}(v, \omega)$ 
0: end for
0: return Published velocity states = 0

```

2.2 Part 2: Environment Mapping

2.2.1 Methodology

The environment is discretized into a 2D occupancy grid where each cell represents a $0.05m \times 0.05m$ area. To handle sensor noise and uncertainty, we employ a probabilistic approach using a **Log-Odds** representation. The log-odds $L(n)$ of a cell n being occupied is updated according to the Bayesian rule:

$$L(n|z_{1:t}) = L(n|z_{1:t-1}) + L(n|z_t) \quad (1)$$

where $L(n|z_t)$ is the inverse sensor model. Using log-odds prevents numerical instability when probabilities approach 0 or 1 and simplifies the update step to an addition. To prevent the map from becoming over-confident and unable to adapt to changes, the values are clamped between $L_{min} = -6.91$ ($p = 0.001$) and $L_{max} = 6.91$ ($p = 0.999$).

For navigation safety, we implement an **Inflation Layer**. Every cell identified as occupied ($p > 0.6$) is expanded by a circular mask with a radius of $0.30m$. This ensures that the RRT* path planner treats the robot as a point mass while maintaining a safe buffer from physical obstacles.

2.2.2 System Architecture and Frame Configuration

The mapping node performs real-time coordinate transformations. Because the LiDAR is offset from the robot center, we use the TF2 buffer to look up the transform T_{map}^{laser} at the exact timestamp of the scan. This allows the system to remain robust during high-speed rotation. The frame naming conventions differ between environments as shown in Table 5.

Table 1: Frame Configuration for Simulation and Real-World Testing

Frame Parameter	Simulation (Gazebo)	Real World / Rosbag
Map Frame (F_{map})	world_enu	odom
Base Frame (F_{base})	base_footprint	base_footprint
Laser Frame (F_{laser})	turtlebot/rplidar	rplidar

The message exchange is summarized in Figure 2.

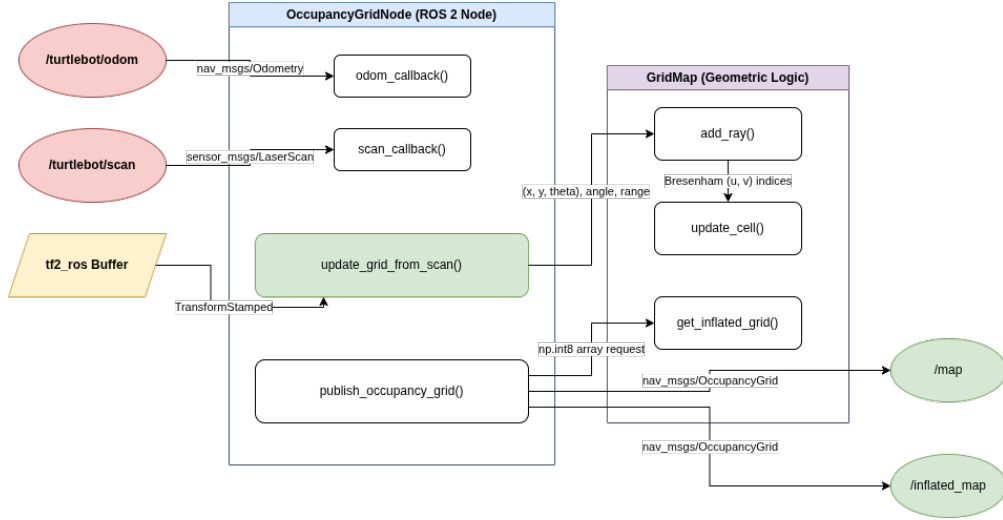


Figure 2: ROS 2 Node and Topic interaction diagram illustrating the flow from LaserScan to the Inflated Map.

2.2.3 Mapping Logic

The mapping process follows a ray-casting approach combined with an inflation post-processing step, as detailed in Algorithm 2.

Algorithm 2 Probabilistic Occupancy Grid Mapping and Inflation

Require: LaserScan data, transform T_{map}^{laser} , occupancy parameters p_{occ}, p_{free} , inflation radius $radius_{inf}$, log-odds limits L_{min}, L_{max}

Ensure: Updated occupancy grid Map_{raw} and inflated map Map_{inf}

```
0: for each LaserScan reception do
0:    $(x_r, y_r, \psi) \leftarrow$  Extract pose from  $T_{map}^{laser}$  lookup
0:   for each beam  $i$  with range  $r_i$  and angle  $\alpha_i$  do
0:      $\beta \leftarrow \psi + \alpha_i$  {Global beam angle}
0:      $(x_{hit}, y_{hit}) \leftarrow (x_r + r_i \cos \beta, y_r + r_i \sin \beta)$ 
0:      $Cells \leftarrow$  BresenhamLine( $(x_r, y_r) \rightarrow (x_{hit}, y_{hit})$ )
0:     for each cell  $c \in Cells$  excluding the last cell do
0:        $G[c] \leftarrow \text{clamp}(G[c] + \log(\frac{p_{free}}{1-p_{free}}), L_{min}, L_{max})$ 
0:     end for
0:     if  $r_i < \text{max\_range}$  then
0:        $G[last] \leftarrow \text{clamp}(G[last] + \log(\frac{p_{occ}}{1-p_{occ}}), L_{min}, L_{max})$ 
0:     end if
0:   end for
0: end for
0:   {Periodic Map Publication (1 Hz)}
0:  $Map_{raw} \leftarrow \text{ConvertLogOddsToProbability}(G)$ 
0:  $Map_{inf} \leftarrow Map_{raw}$ 
0: for each cell  $c$  where  $Map_{raw}[c] > 0.6$  do
0:   Apply circular mask of  $radius_{inf}$  to  $Map_{inf}$ 
0: end for
0: Publish  $Map_{raw}$  and  $Map_{inf}$ 
0: return Updated grid maps =0
```

2.3 Part 3: Path Planning and Control

2.3.1 Methodology

This section implements an online motion planning architecture that tightly couples the occupancy grid map, the RRT* algorithm, and a closed-loop waypoint follower.

The architecture is designed to be **asynchronous**: a high-level path planning loop runs at 1 Hz to handle global trajectory updates, while a low-level velocity controller runs at 10 Hz to maintain precise tracking and react to local pose changes.

RRT* and Path Smoothing The RRT* algorithm is used to find an optimal path from the robot's current position to a designated goal. To ensure the path is executable by a differential drive robot, we apply a **path smoothing** step. This step iterates through the generated waypoints and attempts to connect non-adjacent nodes directly if the segment between them is collision-free, effectively reducing the "zig-zag" nature inherent in RRT-based sampling.

Bisection Collision Checking and Replanning To maintain safety in dynamic environments, we implement a recursive **Bisection Method** for collision checking. Instead of only checking the start and end of a segment, the algorithm recursively evaluates the mid-point of segments down to a maximum depth (d_{max}). This ensures that even thin obstacles are detected.

The replanning mechanism operates by continuously monitoring the segment between the robot and its immediate next waypoint (q_{next}). If the bisection check returns a collision in the latest `/inflated_map`, the robot triggers an emergency stop and the planner is forced to compute a new path from the current pose.

2.3.2 System Configuration

The following tables summarize the parameters used for both the controller and the path planner. The controller gains were adjusted to account for the latency and friction differences between the Gazebo simulation and the physical TurtleBot3 hardware.

Table 2: Motion Controller Configuration (Simulation vs. Real World)			
Parameter	Symbol	Simulation	Real World Testing
Wheel Base Distance	L	0.230 m	0.230 m
Linear Velocity Gain	k_v	1.5	1.2
Angular Velocity Gain	k_w	1.5	1.0
Max Linear Velocity	v_{max}	0.3 m/s	0.2 m/s
Max Angular Velocity	ω_{max}	0.3 rad/s	0.3 rad/s

Table 3: RRT* Path Planner Configuration

Parameter	Symbol	Value
Step Size (Extension)	Δq	8.0 units
Maximum Bisection Depth	max_depth	4
Neighbor Search Radius	$radius$	5.0 units
Goal Bias Probability	p	0.3
Maximum Iterations	$iter_{max}$	2000

2.3.3 System Architecture

The online motion planning architecture consists of two primary nodes: `grid_mapping` and `sampling_turtlebot`. The message exchange between these nodes is summarized in Figure 3.

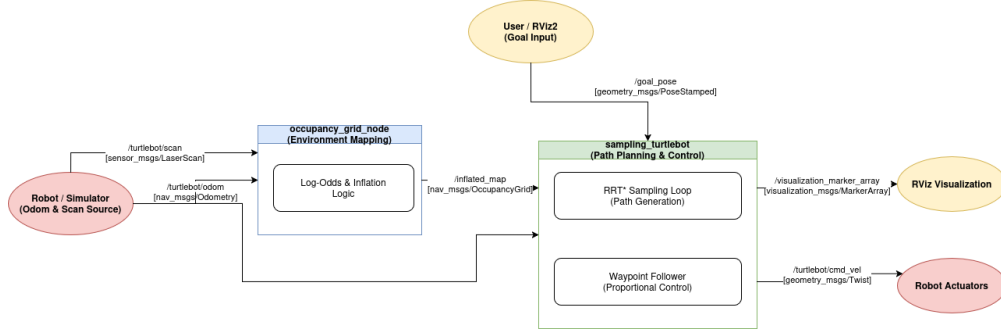


Figure 3: ROS 2 Node and Topic interaction diagram for the navigation stack.

2.3.4 Algorithm Pseudocode

Algorithm 3 Hybrid Path Planning and Velocity Control

```

1: Loop 1: Path Planning (1 Hz)
2:   if Map, Robot Pose, or Goal is missing then
3:     continue
4:   end if
5:   collision  $\leftarrow$  BisectionCheck(RobotPose, NextWaypoint)
6:   if collision or PathIsEmpty then
7:     Stop robot (publish  $v = 0, \omega = 0$ )
8:      $G, edges \leftarrow$  RRT_Star_Sample(start, goal,  $iter_{max}$ )
9:     if Path found then
10:      path  $\leftarrow$  Smoothing(FillPath( $G, edges$ ))
11:      Waypoints  $\leftarrow$  TransformToWorldCoordinates(path)
12:    end if
13:  end if
14: Loop 2: Velocity Control (10 Hz)
15:   if Waypoints is Empty then
16:     Stop robot; return
17:   end if
18:   target  $\leftarrow$  Waypoints[0]
19:   dist,  $\theta_{err} \leftarrow$  CalculateErrors(RobotPose, target)
20:   if dist < AcceptanceRadius then
21:     PopFirstWaypoint(Waypoints)
22:     Stop robot
23:   else
24:      $\omega \leftarrow$  clamp( $k_w \cdot \theta_{err}, \pm \omega_{max}$ )
25:     if  $|\theta_{err}| < 0.3$  rad then
26:        $v \leftarrow$  clamp( $k_v \cdot dist, 0, v_{max}$ )
27:     else
28:        $v \leftarrow 0$  {Pivot in place to minimize angular error}
29:     end if
30:     PublishTwist( $v, \omega$ )
31:   end if
=0

```

3 Results and Discussion

3.1 Part 1: Low-Level Motion Control Node

3.1.1 Controller Parameter Configuration

The performance of the low-level controller is highly dependent on the chosen gains and thresholds. The following table summarizes the key parameters used during the experiments:

Table 4: Controller Configuration Parameters for Part 1		
Parameter	Value	Description
<i>acceptance_radius</i>	0.1	Distance threshold (m) to consider the goal reached.
<i>max_linear_velocity</i>	0.5	Maximum allowed forward speed (m/s) for safety.
k_v	1.0	Proportional gain for linear velocity control.
k_w	2.0	Proportional gain for angular velocity (steering).
$\omega_{threshold}$	0.3	Heading error threshold (rad) below which linear motion is enabled.

The controller's performance is driven by the balance between the proportional gains and the steering threshold. By setting $k_w = 2.0$ and $k_v = 1.0$, the robot achieves responsive orientation and stable approach speeds without excessive oscillation. The threshold $\omega_{threshold} = 0.3$ rad ensures "smooth" motion by enforcing in-place rotation until the heading error is minimized, preventing wide, inaccurate trajectories. Finally, an *acceptance_radius* of 0.1 m provides a reliable stopping condition that accounts for odometry drift while maintaining target precision.

3.1.2 Execution and Deployment Instructions

The control system is designed to run within the ROS 2 Jazzy environment. The deployment process can be executed through two different methods depending on whether the user prefers a graphical interface or direct command-line control.

Method 1: Interactive Navigation via RViz This method allows for intuitive goal setting using the RViz GUI tools.

1. **Launch the Simulator:** Open a terminal and start the TurtleBot simulation environment (Stonefish):

```
ros2 launch turtlebot_simulation kobuki_basic.launch.py
```

2. **Execute Control Node:** Open a second terminal and run the developed motion control node:

```
ros2 run online_motion_planning control_tb
```

3. **Set Goal:** In the RViz interface, use the "2D Goal Pose" tool [cite: 31] to click and drag a target location on the map.

Method 2: Programmatic Navigation via CLI The robot can also be commanded to specific coordinates using the terminal, which is useful for precise testing or automated scripts.

1. Ensure the simulator and control node are running as described in Method 1.
2. **Publish Goal Command:** Open a new terminal and publish a message to the `/goal_pose` topic:

```
ros2 topic pub /goal_pose geometry_msgs/msg/PoseStamped \  
"{header: {stamp: {sec: 0}, frame_id: 'map'}, \  
pose: {position: {x: 2.0, y: 1.0, z: 0.0}}}" --once
```

The robot will automatically calculate the trajectory using the implemented control law and move to the specified (x, y) coordinates without requiring manual input from the RViz GUI.

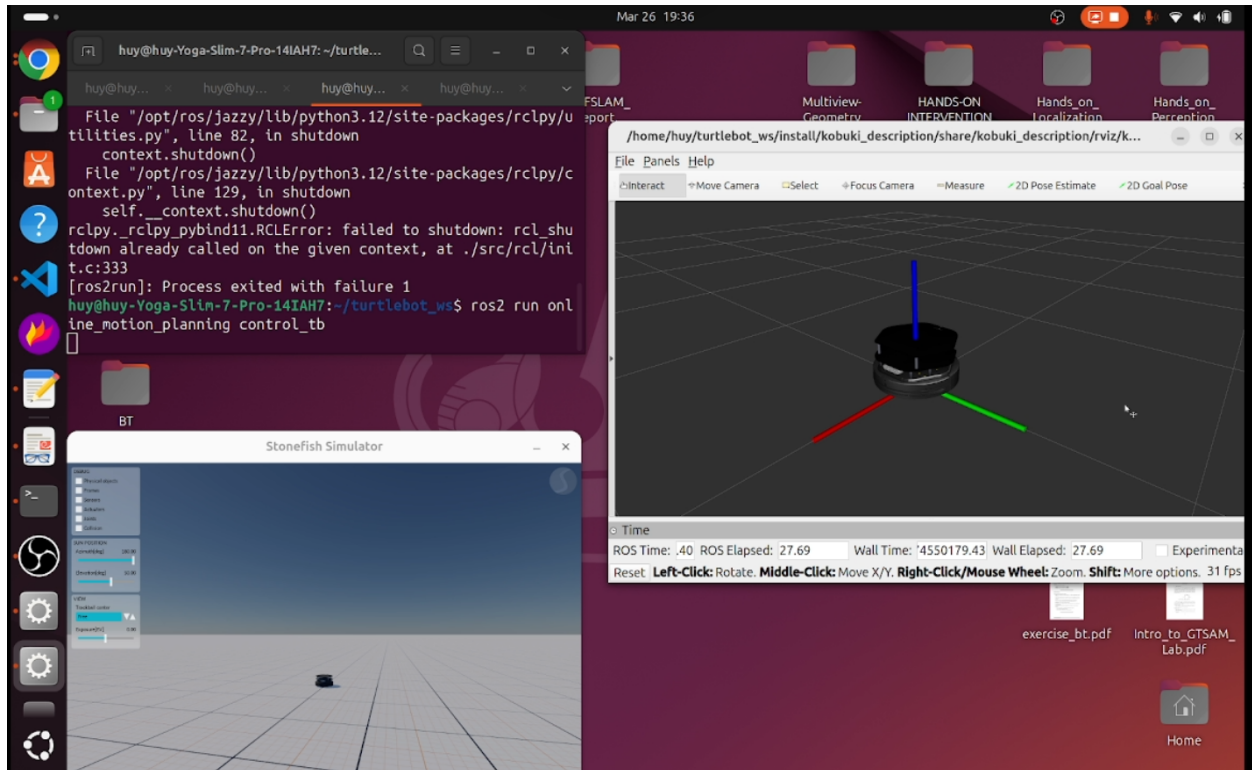


Figure 4: Screen execution of Part 1 low-level motion control node

3.2 Part 2: Environment Mapping

3.3 System Configuration: Coordinate Frames

The mapping node relies on accurate Coordinate Frame (TF) transforms. The following table documents the specific frame names used to distinguish between the Gazebo simulation environment and the physical robot (or rosbag playback).

Table 5: Frame Configuration for Mapping

Frame Parameter	Simulation (Gazebo)	Real World / Rosbag
Map Frame (F_{map})	world_enu	odom
Base Frame (F_{base})	base_footprint	base_footprint
Laser Frame (F_{laser})	turtlebot/rplidar	rplidar

3.3.1 Coordinate Frame Analysis (ENU vs. NED)

During initial testing, a "mirroring" effect was observed in the generated occupancy grids, where the LIDAR scans appeared to rotate in the opposite direction of the robot's physical heading. Through an analysis of the transform tree ($TF2$), it was determined that the discrepancies stemmed from the different orientations of the Z -axis across the two testing platforms.

In the *Standard ROS (ENU)* convention used by the real robot, the Z -axis points upward. Following the right-hand rule, a positive rotation is counter-clockwise. Since the `LaserScan` message also increments counter-clockwise, the global beam angle is additive:

$$\theta_{beam} = \theta_{laser} + \alpha_{scan} \quad (2)$$

Conversely, in the *Simulation (NED)* environment, the Z -axis is flipped (pointing downward). In this configuration, a positive yaw rotation results in a clockwise movement when viewed from above. To align the counter-clockwise laser sweep with this coordinate system, the relative angle must be subtracted to prevent "mirrored" mapping results:

$$\theta_{beam} = \theta_{laser} - \alpha_{scan} \quad (3)$$

3.3.2 Execution and Deployment Instructions

This section provides the procedure for executing the probabilistic occupancy grid mapping node using pre-recorded experimental data (Rosbag).

1. **Rosbag Playback:** Navigate to the directory containing the recorded data and initiate the playback. This broadcasts the `/turtlebot/odom` and `/turtlebot/scan` topics required for mapping:

```
cd src/grid_mapping/bag_files
ros2 bag play part2_real_robot.bag
```

2. **Executing the Mapping Node:** In a separate terminal, execute the occupancy grid node. This node processes the laser scans using a log-odds inverse sensor model and generates the occupancy grid:

```
ros2 run grid_mapping occupancy_grid_original
```

3. **Visualization in RViz2:** To monitor the mapping process in real-time, launch RViz2 and configure the following:

- Set the **Fixed Frame** to `odom`.
- Add a **Map** display and subscribe to the `/map` topic.
- Add a **LaserScan** display for the `/turtlebot/scan` topic to verify alignment.

Implementation Note: The node is configured to use the `odom` frame as the map origin for real-world playback. It publishes two layers: a raw probabilistic map to `/map` and an inflated safety map to `/inflated_map` for use by the RRT* planner.

3.4 Mapping Result

The students carried out the mapping in real-world testing with the Turtlebot. The robot is teleoperated to travel around an enclosed environment while the data of robot scan, odometry is recorded. The mapping node is run on the recorded data. Figure 5 illustrates the mapping generated by the robot and the travelled trajectories. The map shows distinct regions of the walls marked as occupied (black) and free space (white).

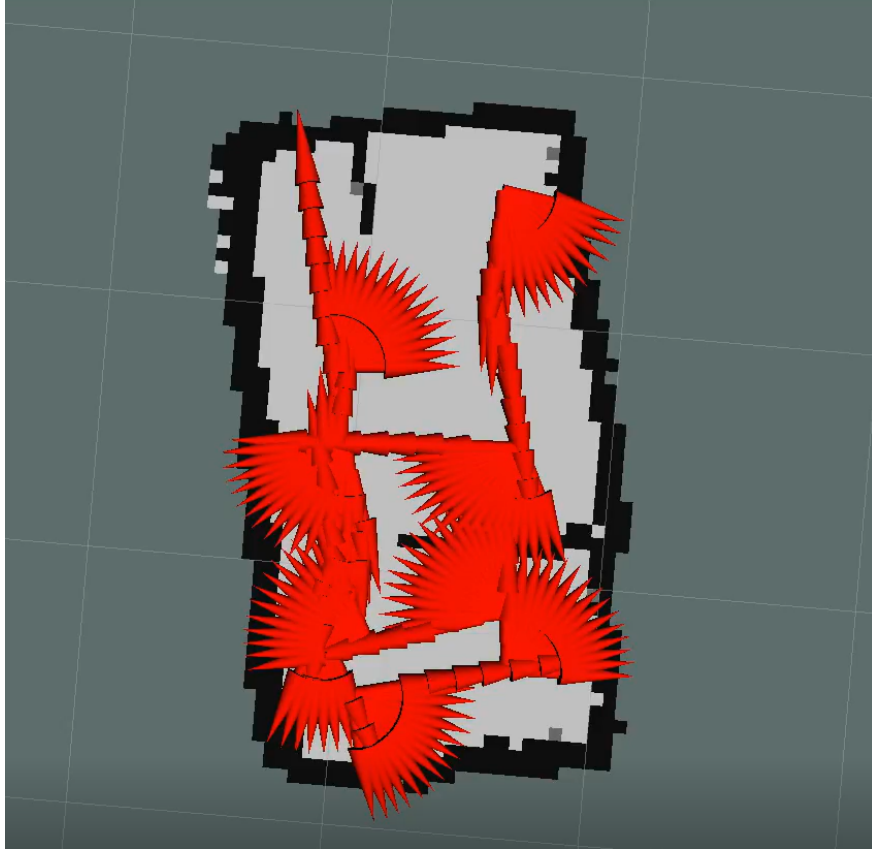


Figure 5: Mapping results of the `grid_mapping` node running on recorded data of the real environment

3.5 Part 3: Path Planning and Navigation

3.5.1 Parameter Configuration

The controller parameters were tuned separately for simulation and hardware. During real-world testing, it was observed that the TurtleBot3 exhibited a significant **mechanical dead-band**; commands below 0.5 m/s failed to overcome static friction. Consequently, the gains and maximum velocities were increased for physical deployment as shown in Table 6.

Table 6: Motion Controller Configuration

Parameter	Symbol	Simulation	Real World Testing
Wheel Base Distance	L	0.330 m	0.230 m
Linear Velocity Gain	k_v	1.0	2.0
Angular Velocity Gain	k_w	1.0	2.0
Max Linear Velocity	v_{max}	0.3 m/s	1.0 m/s
Max Angular Velocity	ω_{max}	0.3 rad/s	1.0 rad/s

Table 7: RRT* Path Planner Configuration

Parameter	Symbol	Value
Step Size (Extension)	Δq	8.0 units
Maximum Bisection Depth	max_depth	4
Neighbor Search Radius	$radius$	5.0 units
Rewire Threshold	$dist_{th}$	5.0 units
Goal Bias Probability	p	0.3

3.5.2 Manual: Operating the Online Motion Planning System

The online motion planning system integrates the occupancy grid mapping, RRT* path planning, and the velocity controller. The following procedures describe how to execute the system in both simulated and physical environments.

To run the motion planning system in a simulated environment (Environment 1), follow these steps:

1. **Launch Simulation Environment:** Execute the dedicated launch file. This starts Gazebo, RViz2, the mapping node, and the RRT* controller node:

```
ros2 launch online_motion_planning \turtlebot_simulation_hoi_circuit_1_rrt.laun
```

2. **Command a Goal:** In RViz2, use the "2D Goal Pose" tool to set a target position. The system will automatically plan a path from the robot's current pose and begin execution.

To deploy the planning stack on the physical TurtleBot3 hardware:

3.5.3 Simulation Results

Experiments were conducted in two distinct environments. Environment 1 features a narrow corridor similar to the laboratory setting, while Environment 2 presents a complex, larger space with multiple obstacles.

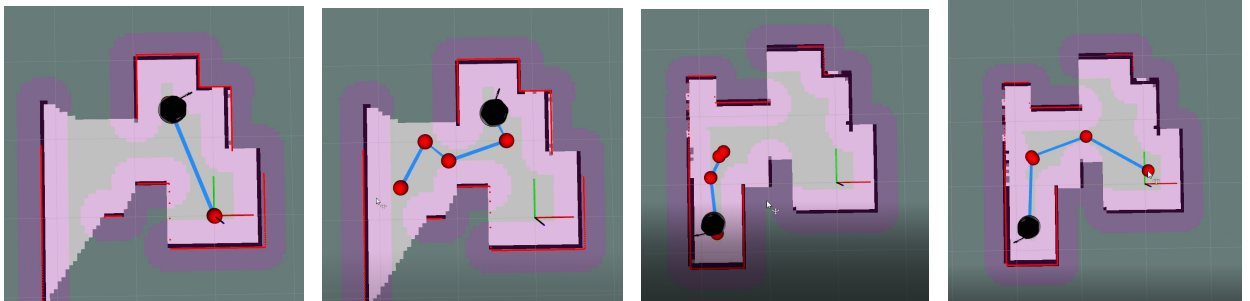


Figure 6: Online motion planning with consecutive goals in Environment 1.



Figure 7: Online motion planning in the complex Environment 2.

In RViz, the black contours represent mapped walls, white indicates free space, and purple represents the inflation layer based on the robot’s footprint. Figure 6 demonstrates that the planner successfully navigates between four consecutive goals. In Environment 2 (Figure 7), the **replanning mechanism** is highlighted: as the robot uncovers previously unexplored obstacles near the far-left corner, the bisection check detects a conflict, halts the robot, and triggers a new collision-free trajectory.

3.5.4 Real World Results

The real-world implementation confirmed the viability of the planning stack. Figure 8 displays the collision-free path generated in the laboratory, while Figure 9 captures the robot’s physical execution.

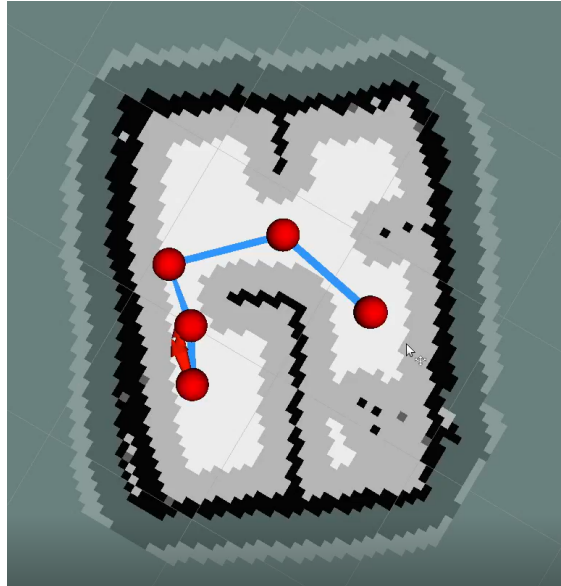


Figure 8: RViz visualization of the planned path during real-world testing.

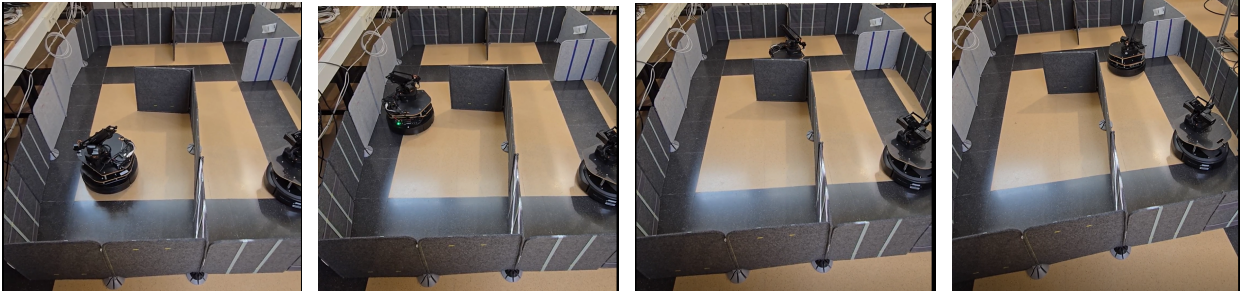


Figure 9: Sequential movement of the TurtleBot3 during the real-world trajectory execution.

The total execution time was approximately 1.34 minutes. We observed intermittent oscillations ("jiggling") and frequent pauses during the trajectory. As noted, the robot requires velocity commands exceeding 0.5 m/s to move reliably. When the controller scales the velocity down (near waypoints or during sharp turns), the robot often oscillate while moving. Future improvements should focus on implementing a minimum velocity threshold or a feed-forward term to compensate for motor friction.

4 Conclusion

The project successfully demonstrated a complete autonomous navigation system on the TurtleBot platform. Starting from a foundational motion controller in Part 1, we established the robot's ability to move accurately. In Part 2, the implementation of a probabilistic occupancy grid allowed the robot to "see" and remember its environment, providing a reliable map despite sensor noise. Finally, Part 3 closed the loop by enabling the robot to plan and execute safe paths through that map using RRT-based algorithms.

Key takeaways from this laboratory include:

- The critical role of **parameter tuning** (controller gains, grid resolution, and RRT step size) in balancing system stability and computational efficiency.
- The effectiveness of the **ROS 2 architecture** in managing high-frequency data exchange between sensing, mapping, and planning nodes.
- The necessity of **dynamic re-planning** to handle the evolving nature of an incrementally built map.

In conclusion, the integrated system fulfills the requirements for autonomous navigation in a moderately challenging environment, proving the robustness of combining probabilistic mapping with sampling-based motion planning.